Machine Learning (2026) 115:58
https://doi.org/10.1007/s10994-025-06992-x

Break a Lag: Triple Exponential Moving Average for Enhanced Optimization

Roi Peleg[1,2] · Yair Smadar[1,2] · Teddy Lazebnik[3,4] · Assaf Hoogi[1,2]

Received: 9 February 2025 / Revised: 10 June 2025 / Accepted: 30 December 2025
© The Author(s) 2026

Abstract
The performance of deep learning models hinges on the effectiveness of their optimization strategies, yet existing methods remain fundamentally constrained. Many optimizers rely on first-order Exponential Moving Average (EMA) techniques, which struggle to accurately track complex gradient trends, resulting in a lag in trend identification and suboptimal convergence. To overcome this critical limitation, we introduce Fast Adaptive Moment Estimation (FAME), a novel optimizer that harnesses the power of Triple Exponential Moving Average (TEMA). By inherently integrating a multi-level tracking mechanism, FAME significantly enhances gradient responsiveness, mitigates trend lag, and maximizes learning efficiency. Our extensive evaluation spans diverse computer vision tasks-including image classification, object detection, and semantic segmentation-incorporating FAME into 22 distinct architectures, from lightweight CNNs to cutting-edge Vision Transformers. Rigorous benchmarking against state-of-the-art optimizers confirms FAME's superiority in accuracy, stability, and robustness. Notably, FAME excels in scalability, demonstrating consistent and substantial performance gains across architectures, tasks, and datasets, positioning it as a powerful optimizer for computer vision.

Keywords Deep learning optimization · Exponential moving average · Trend tracking · Gradient-based optimization

1 Introduction

Optimization techniques are foundational to advancing computer vision, enabling models to learn from data through the minimization of loss functions efficiently. With the increasing complexity of visual tasks–such as object detection, image segmentation, and fine-grained classification, the need for optimizers that strike a delicate balance between computational efficiency and generalization performance has never been more critical. Stochastic Gradient Descent (SGD) (Robbins and Monro, 1951) laid the foundation for modern optimization

Communicated by Lam Nguyen.

Extended author information available on the last page of the article

Published online: 06 March 2026

by uniformly updating parameters based on gradient information. Momentum (Qian, 1999) enhanced SGD by incorporating past gradients, smoothing the update trajectory, and accelerating convergence. However, SGD's computational demands, particularly in the high-dimensional parameter spaces of current vision models, necessitated the development of adaptive learning rate methods that dynamically adjust learning rates based on gradient history (Mulloy, 1994). Adaptive optimization methods introduced a different paradigm by assigning individualized learning rates to each parameter, significantly improving convergence speed in complex loss landscapes. Therefore, current optimization approaches can be broadly categorized into two classes: momentum-based techniques, which focus on stabilizing gradient updates for smoother convergence, and adaptive learning rate methods, which scale updates dynamically based on past gradient magnitudes. AdaGrad (Duchi et al., 2011) pioneered the latter approach by adjusting learning rates based on accumulated gradient information, while RMSProp (Tieleman and Hinton, 2012) mitigated AdaGrad's diminishing learning rate issue through exponential decay. Kingma and Ba (2014) combined the strengths of both by leveraging exponential moving averages (EMA) for moment estimation, refining gradient updates to improve adaptability. Building on these foundations, newer optimizers have introduced refinements to address their respective limitations. AMSGrad (Reddi et al., 2019) stabilizes updates by preventing learning rate increases, (Loshchilov et al., 2017) decouples weight decay for better generalization, and AdaBound (Luo et al., 2019) dynamically constrains learning rates to mitigate excessive adaptation. AdaHessian (Yao et al., 2021) further extends these ideas by incorporating second-order curvature information for better navigation of complex loss surfaces. Despite their rapid convergence, EMA-based adaptive optimizers often struggle with generalization, particularly in deep vision tasks. Conversely, while SGD is widely recognized for its strong generalization ability, recent studies suggest that its effectiveness is largely limited to classical ConvNets and faces challenges when applied to modern architectures (Xie et al., 2024; Yu et al., 2022; Li et al., 2024). Hybrid approaches, such as SWATS (Keskar and Socher, 2017) and Mixing Adam and SGD (MAS) (Landro et al., 2020), aim to bridge this gap by dynamically shifting between optimization strategies. However, they fall short of fully exploiting the strengths of both paradigms.

This persistent challenge highlights the need for a new optimization framework–one that preserves the fast convergence of adaptive methods while enhancing generalization and stability in large-scale vision models. To address this, we introduce FAME and its following key contributions:

- **Advanced High-Order Optimization:** FAME is the first deep learning optimizer to integrate **Triple Exponential Moving Average (TEMA)**, establishing a *hierarchical multi-level structure* that significantly improves optimization precision, stability, and adaptability.
- **Overcoming the Limitations of EMA-based Optimizers:** Traditional adaptive methods often struggle with inaccurate gradient adaptation and instability, particularly in large-scale vision tasks. FAME introduces an advanced tracking mechanism that *strategically balances short-term responsiveness to gradient variations with long-term stability, mitigating the inherent lag of EMA-based approaches*. This enables more precise trend identification and robust optimization dynamics, leading to improved convergence and generalization.

- **Extensive Validation:** Rigorous experiments across *22 model architectures*, spanning diverse vision tasks and five major benchmarks, confirm FAME's *consistent superiority in accuracy, robustness, and computational efficiency* compared to widely used optimizers.

2 FAME: Fast-Adaptive Moment Estimation

2.1 EMA: A Foundation for Trend Analysis

EMA enables temporal data analysis. At its core, EMA implements a theoretically grounded weighting scheme that decays exponentially over time, and balances noise reduction and trend preservation. This mathematical foundation sets it apart from conventional smoothing operators, particularly Gaussian kernels, by minimizing temporal lag while maintaining signal fidelity. Formally, let $x = x_{1:t} \in \mathbb{R}^t$ represents a temporal sequence up to time step *t*. EMA can be elegantly expressed through a recursive formulation:

$$\mathrm{EMA}(x_{1:t}) = \beta \cdot \mathrm{EMA}(x_{1:t-1}) + (1-\beta) \cdot x_t = (1-\beta) \sum_{i=0}^{t} \beta^i x_{t-i}, \tag{1}$$

EMA relies on its smoothing coefficient $\beta \in [0, 1]$, which forces a critical trade-off: small β values allow quick adaptation but amplify noise, while large β values reduce noise but introduce significant delays. This means first-order EMA must always sacrifice either speed or stability.

Although first-order EMA's exponentially decaying memory structure helps stabilize gradient estimates, it fundamentally suffers from two problems: it lags behind actual changes and tends to over-smooth important gradient patterns. These issues can significantly impact optimization performance, especially in complex deep learning tasks.

Higher-order EMA techniques systematically address these limitations. By using multiple levels of exponential smoothing, they can achieve both quick adaptation and noise reduction - capabilities that are mathematically impossible with first-order EMA's single smoothing coefficient (illustrated in Fig. 1).

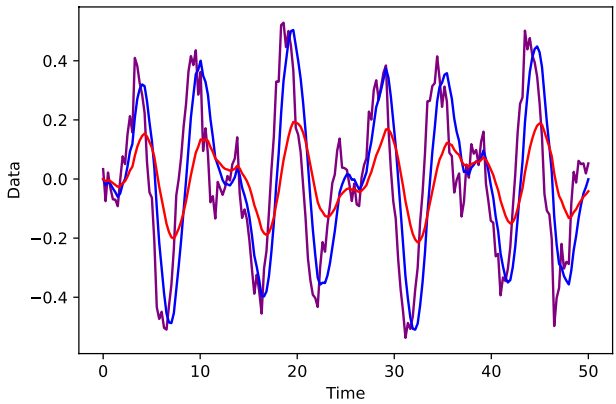

Fig. 1 Simulated demonstration of gradient trend estimation and lagging. Ground truth (GT, purple), triple exponential moving average (TEMA)-based estimation (blue), and exponential moving average (EMA)-based estimation (red)

2.2 Double Exponential Moving Average (DEMA)

We introduce advanced smoothing techniques that address the inherent lag limitations of traditional Exponential Moving Averages (EMA). The Double Exponential Moving Average (DEMA) incorporates a lag-correction mechanism through:

$$\mathrm{DEMA}(x) = 2\mathrm{EMA}_1(x) - \mathrm{EMA}_2(x) \tag{2}$$

where $\mathrm{EMA}_k(x)$ represents k recursive applications of EMA:

$$\mathrm{EMA}_k(x) = \underbrace{\mathrm{EMA}(\mathrm{EMA}(\cdots \mathrm{EMA}(x)))}_{k \text{ times}} \tag{3}$$

The key innovation in DEMA lies in its lag-correction term $[\mathrm{EMA}_1(x) - \mathrm{EMA}_2(x)]$, which provides a robust estimation of the temporal displacement between raw data *x* and its primary smooth estimate $EMA_1(x)$. This formulation enables DEMA to achieve superior temporal responsiveness while preserving signal fidelity.

2.3 Triple Exponential Moving Average (TEMA)

Building upon DEMA's foundation, the Triple Exponential Moving Average (TEMA) introduces a second-order correction mechanism:

$$\mathrm{TEMA}(x) = 3\mathrm{EMA}_1(x) - 3\mathrm{EMA}_2(x) + \mathrm{EMA}_3(x) \tag{4}$$

TEMA formulation emerges from a hierarchical lag correction process:

$$\begin{aligned}
\mathrm{TEMA}(x) &= \mathrm{EMA}_1(x) + [\mathrm{EMA}_1(x) - \mathrm{EMA}_2(x)] \\
&\quad + [(\mathrm{EMA}_1(x) - \mathrm{EMA}_2(x)) - \mathrm{EMA}(\mathrm{EMA}_1(x) - \mathrm{EMA}_2(x))] \\
&= \mathrm{EMA}_1(x) + [\mathrm{EMA}_1(x) - \mathrm{EMA}_2(x)] \\
&\quad + [\mathrm{EMA}_1(x) - 2\mathrm{EMA}_2(x) + \mathrm{EMA}_3(x)] \\
&= 3\mathrm{EMA}_1(x) - 3\mathrm{EMA}_2(x) + \mathrm{EMA}_3(x)
\end{aligned} \tag{5}$$

The formulation consists of three critical components: (1) the initial smooth estimate $\mathrm{EMA}_1(x)$, (2) the first-order lag correction $[\mathrm{EMA}_1(x) - \mathrm{EMA}_2(x)]$, and (3) the second-order refinement term $[\mathrm{EMA}_1(x) - 2\mathrm{EMA}_2(x) + \mathrm{EMA}_3(x)]$. This hierarchical structure enables TEMA to achieve unprecedented adaptability to temporal dynamics while maintaining signal coherence.As visualized in Fig. 2, TEMA's components exhibit distinct phase characteristics that, when combined, achieve optimal lag reduction while preserving signal integrity. The complementary nature of these components enables robust temporal adaptation across diverse signal dynamics.

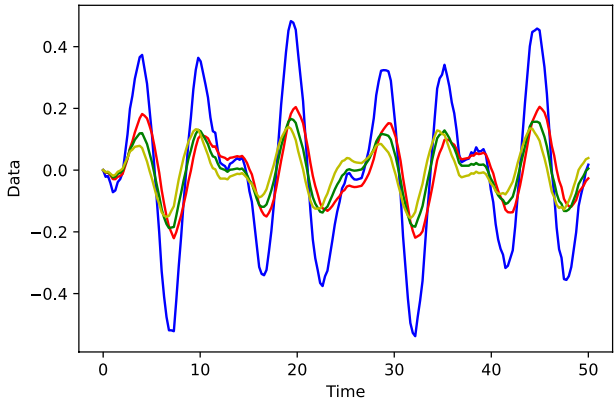

Fig. 2 Decomposition of TEMA (blue) into its constituent components: EMA_1 (red), first-order correction $(\mathrm{EMA}_1 - \mathrm{EMA}_2)$ (green), and second-order correction $(\mathrm{EMA}_1 - 2\mathrm{EMA}_2 + \mathrm{EMA}_3)$ (yellow). The composite TEMA signal demonstrates the synergistic integration of these components

2.4 Generalized k-th Order Exponential Moving Average

We further generalize this framework to arbitrary orders through the k-th order EMA formulation:

$$\mathrm{KEMA}_k(x) = \sum_{i=1}^{k} (-1)^{i+1} \binom{k}{i} \mathrm{EMA}_i(x) \tag{6}$$

This generalization provides fine-grained control over the temporal responsiveness-smoothness trade-off, enabling optimal adaptation to specific optimization scenarios. The binomial coefficients in this formulation ensure proper weighting of higher-order corrections, maintaining mathematical consistency with lower-order variants.

2.5 FAME Intuition

This section provides a rigorous analysis of FAME's underlying mechanisms and its advantages, rooted in mathematical foundations. Exponential Moving Average (EMA) is a core smoothing technique that dynamically weights sequential data, emphasizing recent values while exponentially attenuating the influence of older ones. This adaptive weighting forms a local observation window that captures short-term patterns while maintaining long-term trends. However, when EMA is recursively applied–smoothing an already smoothed signal (EMA(EMA))–the hierarchical compounding effect extends the effective observation window, facilitating more advanced trend extraction. Building upon this principle, FAME introduces three fundamental advancements:

1. **Hierarchical Multi-Scale Optimization**: Traditional EMA-based optimizers face an inherent trade-off: short smoothing windows enable rapid adaptation but amplify noise, while larger windows enhance stability at the cost of responsiveness. FAME overcomes this limitation through a hierarchical triple-EMA (TEMA) structure, forming a cascading filtering pyramid. The first level captures immediate gradient fluctuations, the second level refines short-term variations and consolidates emerging trends, and the third level enforces long-term stability, mitigating premature convergence and excessive

oscillations. This multi-scale architecture enables FAME to swiftly adapt to gradient changes while maintaining robust convergence, outperforming conventional first-order EMA-based optimizers.

2. **Adaptive Trend Identification**: Standard EMA-based optimizers react passively to past gradients, often struggling to detect new optimization patterns in real time. FAME's TEMA mechanism integrates a three-stage verification process in which the first level detects potential shifts, the second level validates their consistency, and the third level confirms their significance. This hierarchical verification enables FAME to distinguish meaningful optimization signals from transient fluctuations, accelerating adaptation and improving convergence robustness.
3. **Resilient Error Correction**: First-order EMA-based optimizers exhibit slow error recovery due to their constrained smoothing capacity. FAME mitigates this limitation through its recursive multi-level filtering, where erroneous updates at one level are dynamically compensated by higher-order corrections that refine subsequent estimates. This recursive adjustment rapidly neutralizes incorrect updates, preventing instability and enhancing recovery efficiency. By continuously recalibrating its optimization trajectory, FAME ensures sustained performance even under noisy or rapidly changing gradient landscapes.

FAME's hierarchical TEMA-based framework fundamentally enhances deep learning optimization, offering a novel synergy between adaptive responsiveness, precise trend detection, and robust error mitigation. This enables fast adaptation to gradient shifts while preserving long-term stability, establishing FAME as a superior alternative to traditional EMA-based methods.

2.6 Formulation of FAME

Let $f(\theta)$ be a differentiable stochastic scalar function that depends on the parameters θ. It is a noisy version of the expected objective function $F(\theta) = \mathbb{E}[f(\theta)]$. Our goal is to minimize $F(\theta)$ with respect to its parameters θ, given only a sequence $f_1(\theta), \ldots, f_T(\theta)$ of realizations of the stochastic function f at subsequent time steps $1, \ldots, T$. The stochasticity in f may originate from evaluations on random subsamples of data points or from inherent function noise.

2.6.1 Gradient Estimation

The gradient, i.e., the vector of partial derivatives of f_t with respect to θ, evaluated at time step t, is denoted by:

$$g_t = \nabla_\theta f_t(\theta) \tag{7}$$

2.6.2 Moment Estimation

Similar to Adam, FAME tracks the first-order and second-order moments (m_t, v_t) of the gradients using an exponential moving average (EMA):

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \end{aligned} \tag{8}$$

where the hyperparameters $\beta_1, \beta_2 \in [0, 1)$ control the exponential decay rates of these moving averages.

FAME extends beyond traditional optimizers by incorporating higher-order moment estimations. It tracks additional variables (dm_t, dv_t) for estimating EMA_2:

$$\begin{aligned} dm_t &= \beta_3 dm_{t-1} + (1-\beta_3) m_t \\ dv_t &= \beta_4 dv_{t-1} + (1-\beta_4) v_t \end{aligned} \tag{9}$$

Based on (dm_t, dv_t), FAME further estimates EMA_3 using variables (tm_t, tv_t):

$$\begin{aligned} tm_t &= \beta_5 tm_{t-1} + (1-\beta_5) dm_t \\ tv_t &= \beta_4 tv_{t-1} + (1-\beta_4) dv_t \end{aligned} \tag{10}$$

Here, (m_0, v_0), (dm_0, dv_0), and (tm_0, tv_0) are all initialized to 0.

Incorporating Eqs. (8–10) into the Triple Exponential Moving Average (TEMA) equation (Eq. 4) yields:

$$\begin{aligned} m_{\mathrm{FAME}_t} &= 3m_t - 3dm_t + tm_t \\ v_{\mathrm{FAME}_t} &= 3v_t - 3dv_t + tv_t \end{aligned} \tag{11}$$

2.6.3 Parameter Update Rule

The final parameter update equation for FAME is given by:

$$\theta_t = \theta_{t-1} - \alpha \cdot \frac{m_{\mathrm{FAME}_t}}{\sqrt{v_{\mathrm{FAME}_t} + \epsilon}} \tag{12}$$

where α is the learning rate and ϵ is a small constant to prevent division by zero.

In our experiments, we assigned the following values for the hyper-parameters: $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$. These values are commonly used by Adam in literature. We also selected $\beta_3 = 0.3$, $\beta_4 = 0.7$, $\beta_5 = 0.8$ and $\epsilon = 1e-8$ for our FAME. These values were chosen empirically as they supplied the best results. The FAME's pseudo-code is provided in Algorithm 1.

Require: Hyper-parameters: $\beta_1, \beta_2, \beta_3, \beta_4, \beta_5, \alpha, \epsilon$
Ensure: Optimized parameters $\hat{\theta}_t$ after convergence
Initialize $m_0 \leftarrow 0$, $v_0 \leftarrow 0$, $dm_0 \leftarrow 0$, $dv_0 \leftarrow 0$, $tm_0 \leftarrow 0$, $tv_0 \leftarrow 0$, $t \leftarrow 0$
while θ_t not converged **do**
  $t \leftarrow t + 1$
  $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ ▷ First moment estimate
  $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ ▷ Second moment estimate
  $dm_t \leftarrow \beta_3 dm_{t-1} + (1 - \beta_3) m_t$ ▷ First moment difference
  $dv_t \leftarrow \beta_4 dv_{t-1} + (1 - \beta_4) v_t$ ▷ Second moment difference
  $tm_t \leftarrow \beta_5 tm_{t-1} + (1 - \beta_5) dm_t$ ▷ Triple moment for first estimate
  $tv_t \leftarrow \beta_4 tv_{t-1} + (1 - \beta_4) dv_t$ ▷ Triple moment for second estimate
  $m_{\mathrm{FAME}_t} \leftarrow 3m_t - 3dm_t + tm_t$ ▷ FAME first moment estimate
  $v_{\mathrm{FAME}_t} \leftarrow 3v_t - 3dv_t + tv_t$ ▷ FAME second moment estimate
  $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \frac{m_{\mathrm{FAME}_t}}{\sqrt{v_{\mathrm{FAME}_t}} + \epsilon}$ ▷ Update rule
end while
return θ_t

Algorithm 1 FAME Optimizer

3 Experiments

We validated the FAME optimizer across *five diverse benchmarks* and *22 model architectures* with varying complexities, covering tasks like detection, classification, and semantic understanding. Importantly, we compared our FAME optimizer to *Adam, SGD, and AdamW as they are the most widely used and well-established baselines in deep learning.* SGD serves as the fundamental optimizer with strong generalization properties, while Adam introduces adaptive learning rates and is the default choice for many architectures. AdamW further improves weight decay handling, making it a strong benchmark. The literature indicates that most alternative optimizers are merely refinements of Adam and SGD, typically yielding only marginal improvements (at best scenario). As a result, additional comparisons would offer limited new insights and are, therefore, redundant. Furthermore, prior literature primarily benchmarks against these three, ensuring our results remain comparable to existing studies.

3.1 Experimental Data

We tested our proposed FAME on five public benchmarks: CIFAR-100 (Krizhevsky, 2009), MS-COCO (Lin et al., 2014), PASCAL-VOC (Everingham et al., 2015), Cityscapes (Cordts et al., 2016), and the large scale ImageNet (Russakovsky et al., 2015).

3.2 Implementation Details

Four key methodological choices were carefully implemented to isolate the optimizer's true impact and ensure comparisons that are fully controlled, fair, rigorous, and transparent:

First, the optimizer was the only variable altered across experiments; all other components of the training pipeline–including model architecture and training procedures–were held strictly constant. This design eliminates confounding factors and ensures that any minor deviations from literature-reported results–arising from weight initialization or inherent stochastic dynamics–impact all optimizers.

Second, all models were trained from scratch, deliberately avoiding pretrained weights. This enables a pure and unbiased assessment of the influence of each optimizer on model performance and generalization. While this choice may lead to slightly lower absolute accuracy compared to pretrained settings, it provides a pure evaluation of the optimizer behavior.

Third, we adhered to the optimal hyperparameters reported in the literature for baseline optimizers, ensuring that our baseline results are consistent with established practice. Minor variations (compared with literature) are naturally expected due to the inherent stochasticity of training and weight initialization.

Fourth, we used identical random seeds across all optimizers for each model, rigorously controlling for randomness and ensuring that all observed differences in performance are attributable solely to the optimizer choice.

4 Results

In pursuit of a comprehensive analysis, we computed several statistical metrics, including accuracy, mAP (mean Average Precision), precision, recall, and F1-score. This involved evaluating the average and standard deviation across three distinct model initializations, solidifying the robustness of our method's validation.

4.1 Image Classification

4.1.1 CIFAR-100 Benchmark

FAME demonstrates superior performance on CIFAR-100 across 17 diverse models (Table 1). When compared with commonly used optimizers, FAME achieved superior accuracy in 84.6% of the models, consistently outperforming existing methods with averaged accuracy improvements of 1.16% over AdamW, 1.57% over AdaBound, 3.52% over AdaHessian, and 16.33% over AdaGrad.

Beyond accuracy gains, FAME demonstrated superior training stability across diverse models. For CNN-based models, FAME reduced epoch-wise accuracy variance by 29.27% compared with existing optimizers (Adam, SGD, AdamW, AdaBound, AdaHessian, AdaGrad). This stability advantage persisted in transformer-based models, where FAME achieves a 21.54% variance reduction compared with the same baseline optimizers. Such consistent stability across different models highlights FAME's robustness to gradient dynamics, effectively mitigating the training fluctuations common in current methods.

4.1.2 ImageNet Benchmark

Our ImageNet experiments reveal FAME's compelling advantages across models. For CNN-based models, FAME achieved consistent accuracy improvements of 0.8% over Adam and 0.65% over SGD. The gains were even more pronounced in Transformer-based models, where FAME surpassed Adam by 0.97% and SGD by 1.27%. Beyond accuracy improvements, FAME enhanced the training process in two critical aspects. First, it provided superior training stability, reducing accuracy fluctuations variance by 5.64% compared with SGD, Adam, and AdamW. Second, it accelerated convergence significantly, reaching 90% of peak

Table 1 Performance comparison of various models using different optimization techniques on CIFAR-100 and ImageNet datasets

Dataset	Type	Model	Our FAME	Adam	SGD + Momentum
CIFAR-100	CNN	EfficientNet-B0 (Tan and Le, 2019)	$0.761_{0.008}$	$0.749_{0.015}$	$0.748_{0.017}$
		EfficientNet-B3 (Tan and Le, 2019)	$0.793_{0.010}$	$0.776_{0.016}$	$0.771_{0.017}$
		MobileNetV3-Large (Howard et al., 2019)	$0.756_{0.005}$	$0.726_{0.007}$	$0.744_{0.011}$
		DenseNet-121 (Huang et al., 2017)	$0.759_{0.008}$	$0.738_{0.013}$	$0.743_{0.012}$
		DenseNet-201 (Huang et al., 2017)	$0.761_{0.005}$	$0.768_{0.006}$	$0.749_{0.006}$
		SE-ResNet-50 (Hu et al., 2018)	$0.772_{0.007}$	$0.761_{0.009}$	$0.757_{0.011}$
		Pyramid-Net-272 (Han et al., 2017)	$0.832_{0.006}$	$0.811_{0.008}$	$0.808_{0.008}$
	Transformer	ViT-Base (Dosovitskiy et al., 2020)	$0.743_{0.003}$	$0.739_{0.002}$	$0.775_{0.006}$
		Swin-T (Liu et al., 2021)	$0.774_{0.002}$	$0.769_{0.007}$	$0.762_{0.002}$
		CvT-13 (Wu et al., 2021)	$0.795_{0.001}$	$0.794_{0.001}$	$0.780_{0.005}$
		T2T-ViT-14 (Yuan et al., 2021)	$0.781_{0.003}$	$0.783_{0.001}$	$0.806_{0.002}$
ImageNet	CNN	MobileNetV3-Large (Howard et al., 2019)	$0.729_{0.001}$	$0.726_{0.001}$	$0.727_{0.001}$
		EfficientNet-B0 (Tan and Le, 2019)	$0.785_{0.016}$	$0.772_{0.014}$	$0.774_{0.016}$
		EfficientNet-B5 (Tan and Le, 2019)	$0.841_{0.016}$	$0.832_{0.014}$	$0.837_{0.016}$
	Transformer	CvT-13 (Wu et al., 2021)	$0.824_{0.009}$	$0.820_{0.010}$	$0.819_{0.016}$
		CAFormer-S18 (Yu et al., 2024)	$0.848_{0.010}$	$0.835_{0.013}$	$0.833_{0.012}$
		CAFormer-S36 (Yu et al., 2024)	$0.869_{0.010}$	$0.857_{0.013}$	$0.851_{0.012}$

The bold values means 1st place in performance

The mean values over 3 different initializations are in standard font, and std values are presented as subscripts

accuracy while using only 78% of the epochs required by other optimizers. This comprehensive enhancement in accuracy, stability, and training efficiency establishes FAME as a powerful optimization solution for large-scale vision tasks.

4.2 Object Detection

4.2.1 MS-COCO Benchmark

Table 2 demonstrates FAME's effectiveness on the MS-COCO benchmark using YOLOv5-S model, where it consistently outperforms standard optimizers including SGD, Adam, and AdamW. FAME achieved a mean Average Precision (mAP@0.5) of 0.569, surpassing SGD (0.549), Adam (0.265), and AdamW (0.446), and also excelled in mAP@0.5:0.95 with a score of 0.375 compared to 0.352 for SGD, 0.211 for Adam, and 0.271 for AdamW. In terms of Precision, FAME scored 0.663, indicating a high proportion of true positives, while Recall was at 0.521, showcasing its effectiveness in capturing relevant objects. Additionally, FAME's F1-Score of 0.583 reflects a strong balance between precision and recall, significantly outperforming SGD (0.518), Adam (0.275), and AdamW (0.471). Overall, these results affirm FAME's enhanced object detection capabilities, positioning it as a robust optimization method in comparison to established techniques.

4.2.2 Pascal-VOC Benchmark

Table 3 presents the analysis of the Pascal Visual Object Classes (VOC) dataset for detection and classification tasks, highlighting FAME's advantages over conventional optimizers like SGD, Adam, and AdamW. For the YOLOv5-S model, FAME achieved a mean Average Precision (mAP) of 0.812, which exceeds the performance of SGD (0.787), Adam (0.651), and AdamW (0.801). Similarly, FAME recorded an impressive mAP of 0.851 for the YOLOv5-m model, compared with SGD (0.828), Adam (0.662), and AdamW (0.83). In classification using the RevViT model, FAME reached an AUC score of 0.696, surpassing SGD (0.679), Adam (0.637), and AdamW (0.671). These findings demonstrate that FAME consistently enhances performance in both detection and classification tasks, underscoring its efficacy as an optimization technique relative to commonly used methods.

Table 2 Object detection results on MS-COCO using YOLOv5-s (Li et al., 2023) and different optimization techniques

Metric	Our FAME	SGD	Adam	AdamW
mAP@0.5	$\mathbf{0.569}_{0.003}$	$0.549_{0.005}$	$0.265_{0.021}$	$0.446_{0.013}$
mAP@0.5:0.95	$\mathbf{0.375}_{0.003}$	$0.352_{0.005}$	$0.211_{0.021}$	$0.271_{0.014}$
Precision	$\mathbf{0.663}_{0.009}$	$0.658_{0.011}$	$0.332_{0.013}$	$0.542_{0.011}$
Recall	$\mathbf{0.521}_{0.006}$	$0.427_{0.006}$	$0.234_{0.019}$	$0.417_{0.007}$
F1-Score	$\mathbf{0.583}_{0.008}$	$0.518_{0.012}$	$0.275_{0.014}$	$0.471_{0.004}$

The bold values means 1st place in performance

The mean accuracy values over 3 different initializations are in standard font, and std values are presented as subscripts. The results match those of training from **scratch**, as reported in the literature

Table 3 Pascal Visual Object Classes (VOC) for detection/classification tasks using various models and different optimization techniques

	Our Fame	SGD	Adam	AdamW
Yolov5-s (Li et al., 2023) (mAP@0.5) - *Detection*	$\mathbf{0.812}_{0.007}$	$0.787_{0.007}$	$0.651_{0.009}$	$0.801_{0.009}$
Yolov5-m (Sadiq et al., 2022) (mAP@0.5) - *Detection*	$\mathbf{0.851}_{0.002}$	$0.828_{0.007}$	$0.702_{0.011}$	$0.830_{0.009}$
ReViT (Diko et al., 2024) (AUC) - *Classification*	$\mathbf{0.696}_{0.006}$	$0.679_{0.007}$	$0.637_{0.009}$	$0.671_{0.007}$

The bold values means 1st place in performance

The mean values over 3 different initializations are in standard font, and std values are presented as subscripts. The results match those for training from scratch as reported in literature

Table 4 Cityscapes semantic segmentation. Performance comparison of various deep models using different optimization techniques

Model	Our FAME	Adam	SGD
DL-V3+ResNet50*	$0.806_{0.009}$	$0.803_{0.011}$	$0.769_{0.013}$
DL-V3+ResNet101*	$0.798_{0.009}$	$0.785_{0.014}$	$0.763_{0.012}$
DL-V3+MobileNetV2*	$0.730_{0.013}$	$0.723_{0.014}$	$0.667_{0.018}$
HANet-ResNet101*	$0.836_{0.001}$	$0.833_{0.001}$	$0.817_{0.013}$
DL-V3+Xception	$0.708_{0.002}$	$0.703_{0.001}$	$0.679_{0.008}$

Cityscapes benchmark. "DL" - DeepLab (Chen et al., 2018). Comparison of classification mIoU (Mean ± Std) supplied by different optimizers across models. **Best results for each model are bolded**. All SGD and Adam results are comparable to the literature results when training from **scratch**. The mean values over 3 different initializations are in standard font, and std values are presented as subscripts. * indicated pretrained on ImageNet to show FAME's strength also in pretrained scenarios, otherwise - training from scratch (DL-V3+Xception)

4.3 Semantic Segmentation

Our evaluation of the Cityscapes benchmark demonstrates FAME's consistent superiority across diverse models (Table 4). FAME achieved for all tested models, with improvements ranging from 0.003% to 1.3% over Adam and 2.5% to 6.3% over SGD. The results also reveal FAME's stability advantage, evidenced by consistently lower standard deviations across three initializations.

4.4 Robustness across Datasets, models, and Weight Initializations

Tables 1, 2, 3 and 4 show that for 87.16% of the models, FAME outperforms all the other optimizers examined in this study, and its performance is comparable with those of the other optimizers for the remaining 12.9% cases. Thus, its high robustness within and across datasets was established. Furthermore, FAME demonstrated a lower standard deviation across the different weight initializations.

Table 5 Effect of the EMA order on the model's performance

Dataset	Model	EMA	DEMA	Our FAME
CIFAR-100	ResNet34	$0.721_{0.008}$	$0.727_{0.006}$	$0.736_{0.006}$
MS-COCO	YOLOv5-s (Li et al., 2023)	$0.265_{0.021}$	$0.317_{0.004}$	$0.569_{0.013}$
PASCAL-VOC	YOLOv5-m (Sadiq et al., 2022)	$0.662_{0.011}$	$0.806_{0.005}$	$0.851_{0.006}$
Cityscapes	DL-V3+ResNet50*	$0.743_{0.008}$	$0.751_{0.005}$	$0.757_{0.005}$

Comparison of model performance (Mean and Std) supplied by different optimizers across datasets. **Best results for each dataset are bolded**. The mean values over 3 different initializations are in standard font, and std values are presented as subscripts

Table 6 Ablation study

Dataset	Adam (Original EMA)	Partial FAME	Our FAME
CIFAR-100 (***Accuracy***)	$0.721_{0.008}$	$0.729_{0.005}$	$0.736_{0.006}$
MS-COCO (***mAP***)	$0.265_{0.021}$	$0.504_{0.006}$	$0.569_{0.013}$
PASCAL-VOC (***mAP***)	$0.662_{0.011}$	$0.829_{0.008}$	$0.851_{0.006}$
CityScapes (***mean IoU***)	$0.743_{0.008}$	$0.751_{0.005}$	$0.757_{0.005}$

The bold values means 1st place in performance

"Partial FAME" means using TEMA for M_{FAME_T} only, while "Our FAME" means using TEMA for both M_{FAME_T} and V_{FAME_T}. CIFAR100: ResNet50, MS-COCO: YOLOv5-s, Pascal-VOC: YOLOv5-m, and Cityscapes: ResNet50+ DeepLabV3

4.5 Ablation Study

- **Effect of High-Order EMAs**—Table 5 shows that increasing the order of the EMA from simple EMA to DEMA, and then to TEMA, leads to improved model performance. Although the 4th order was also tested, it underperformed compared to TEMA, which balanced between trend estimation, smoothness and lag reduction. The results emphasize that our triple-based FAME consistently outperformed other high-order EMAs across various datasets, tasks, and models, delivering superior accuracy.
- **Effect of TEMA on Different Moments**—Table 6 illustrates how incorporating TEMA into the 1st and 2nd moments of FAME enhanced its performance. The shift from EMA to Partial TEMA (for 1st m_{FAME_t} moment only) and then to full FAME (for both m_{FAME_t} and v_{FAME_t} moments) consistently boosted accuracy. This demonstrates FAME's capability to improve optimization across object detection, classification, and segmentation tasks, regardless of the dataset or model.

4.6 Sensitivity to Hyper-parameters Tuning

4.6.1 FAME's Hyper-parameter Robustness

Although FAME introduces three additional hyperparameters (β_3, β_4, β_5) beyond Adam's β_1 and β_2, it demonstrates consistently higher robustness to varied hyperparameter settings.

For **Adam**, β_1 and β_2 were independently swept over the range [0.9, 0.99], which encompasses their values recommended in the literature. We used a 9×9 grid with 0.01 increments to include all optional combinations of β_1 and β_2. Following best known practices, Adam's hyperparameters were then fixed to rigorously isolate and assess the sensitivity of the additional parameters of FAME. We conducted independent sweeps of FAME's β_3, β_4, and β_5 in a substantially broader and more challenging range (compared to Adam's) of [0.1, 0.9], using a $9 \times 9 \times 9$ grid with 0.1 increments. This more extensive exploration further underscores the inherent stability of FAME in diverse hyperparameter configurations. Evaluated on example models from three main benchmarks, FAME exhibited only 3.72% performance variation across all models and datasets tested, compared to 10.12% supplied by Adam (Table 7 in the Supplementary material). Importantly, FAME's hyperparameters were fixed after choosing their optimal combination and were used consistently in all tasks, eliminating the need for per-task adjustment and further highlighting the practical robustness of FAME.

4.6.2 General Deep Model's Hyper-parameters

While we initially adopted hyperparameter values from the literature, we conducted a further focused grid search around these well-explored defaults in selective representative models, systematically exploring neighboring configurations for all core hyperparameters, including batch size, learning rate, momentum, weight decay, and number of epochs. This rigorous exploration provided a robust and consistent foundation for our evaluation.

4.7 Memory Cost and Computational Efficiency

FAME introduces only a marginal per-epoch overhead of 3.7% in memory usage and 2.85% in computation time, relative to Adam's 1st-order *EMA* baseline (averaged across all tested models). These values were measured on an NVIDIA A100 GPU.

Importantly, this minimal overhead is more than offset by the superior training efficiency of FAME that requires only 76.3% of the total training epochs that Adam typically needs to achieve convergence. As a result, FAME not only preserves comparable memory and computational efficiency but also delivers substantially faster convergence.

As illustrated in Table 8 (Supplementary Material), these trends remain consistent across a broad range of models and datasets.

4.8 Convergence Analysis

To demonstrate the convergence of the proposed FAME algorithm, we begin by establishing a mathematical framework. Let $d \in \mathbb{N}$ represent the number of parameters in the function $F : \mathbb{R}^d \to \mathbb{R}$ that we aim to optimize. In the domain of machine learning, F embodies the complete training objective function, and optimization algorithms are employed to locate critical points within F. Specifically, our focus lies on optimization techniques that expand upon the classical Gradient Descent (GD) algorithm by integrating a heavy-ball style momentum parameter (Polyak, 1964). These algorithms exhibit a continuously diminishing step size from an initial finite step, leading to a monotonically decreasing process (Kingma and Ba, 2014). This behavior underscores their progression towards convergence.

Let us assume several constraints: $0 \leq \beta_5 \leq \beta_3 \leq \beta_1 \leq 1$ and $0 \leq \beta_4 \leq \beta_2 \leq 1$, along with the condition $\beta_1 \leq \beta_2$, and a non-negative value for α.

According to Algorithm 1, we define three sequences, $m_i, v_i, \theta_i \in \mathbb{R}^n$, where n denotes the dimension of the parameter space. Given an initial point $\theta_0 \in \mathbb{R}^n$ and setting $m_0 = v_0 = 0$, we proceed under three primary assumptions:

- The loss function F is bounded below by an arbitrary function F^* for any point $\forall \theta \in \mathbb{R}^n : F^*(\theta) \leq F(\theta)$.
- The loss function, concerning the max-norm (l_∞), is uniformly and surely bounded: $\forall \epsilon > 0, \exists r > \epsilon \forall \theta \in \mathbb{R}^n : ||\nabla F||_\infty \leq r - \epsilon$.
- The loss function maintains L-Lipschitz continuity concerning the l_2-norm: $\forall \theta, \zeta \in \mathbb{R}^n : ||\nabla F(\theta) - F(\zeta)||_2 \leq L||\theta - \zeta||_2$.

For a specific number of iterations $N \in \mathbb{N}$, we introduce τ as a random index ranging from $\{0, \ldots, N-1\}$, where $\forall i \in \mathbb{N} : i < N, P[\tau = i] \propto 1 - \beta_1^{N-i}$. This implies that for $\beta_1 = 0, \tau$ is uniformly sampled, while higher orders approaching zero lead to fewer samples in later iterations. This stratagem aims to limit the expected squared gradient norm at iteration τ.

For $N > \beta_1/(1-\beta_1) \in \mathbb{N}$, the following inequality holds:

$$\begin{aligned} ||\nabla F(\theta_\tau)||^2 &\leq 2r\sqrt{N}\frac{F(\theta_0) - F^*}{\alpha(N - \frac{\beta_1}{1-\beta_1})} \\ &+ \frac{\sqrt{N}}{N - \frac{\beta_1}{1-\beta_1}} \ln\left(1 + \frac{Nr^2}{\epsilon}\right)\left(\alpha n r L + \frac{12dR^2}{1-\beta_1} + \frac{2\alpha^2 d L^2 \beta_1}{1-\beta_1}\right) \end{aligned} \tag{13}$$

Continuing with Algorithm 1, we can split the proof into two cases:

- When $\beta_3 = \beta_5 = \beta_4 = 0$, the algorithm adopts the form $\theta_t = \theta_{t-1} - \alpha \frac{m_t}{v_t}$, akin to the AdaGrad algorithm, known for convergence (Defossez et al., 2022).
- For $\exists j \in \{3, 4, 5\} : \beta_j > 0$, dm_t, tm_5, dv_t, and tv_t depend on m_t and v_t, converging at some point following the first case. Thus, a $\delta \in \mathbb{N}$ emerges with $\epsilon \|\psi_\delta - \psi_{\delta-1}\|_2 \leq \epsilon^*$ for $\psi \in \{dm, tm, dv, tv\}$. Consequently, beyond this interaction (δ), the update rule transforms into $\forall t > \delta : \theta_t = \theta_{t-1} - \alpha \frac{m_t + c_1}{v_t + c_2}$, with $c_1, c_2 \in \mathbb{R}^n$. This update rule can be upper-bounded by a constant $c_3 \in \mathbb{R}^n$, resembling ADAgrad, because the adaptive update roughly aligns with the descent direction. Thus, the convergence at $\tau \in \mathbb{N}$ implies subsequent steps being smaller, bounded by ADAgrad convergence processes from the same initial configuration.

Ultimately, by establishing an upper limit to convergence in the second case, we demonstrate convergence in this scenario as well.

5 Conclusion

FAME represents a substantial advancement in core optimization algorithms, leveraging TEMA to effectively address the well-known limitations of EMA-based optimizers in gradient tracking and optimization lag. In comprehensive evaluations spanning 22 diverse architectures across classification, detection, and dense prediction tasks, FAME demonstrated consistent superiority – achieving state-of-the-art performance in 88.46% of all models' experiments while maintaining exceptional stability. Critically, FAME delivered a 23.7% reduction in the total number of training epochs while maintaining robust performance across vastly different architectures and benchmarks. This combination of superior accuracy, high robustness, and faster convergence, grounded in the hierarchical EMA structure of FAME, establishes FAME as a powerful optimizer that contributes to the theoretical understanding of optimization dynamics and delivers substantial practical improvements.

While FAME exhibits significant advantages over previous optimizers, its triple EMA structure adds a $\sim 3\%$ per-epoch overhead in memory and computation. Though this overhead is generally mitigated by its accelerated convergence, it could potentially present a per-epoch challenge in extremely resource-constrained settings.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/s10994-025-06992-x.

Author Contributions R.P. contributed to methodology, software development, formal analysis, investigation, data curation, and visualization, as well as manuscript review and editing. Y.S. was involved in software development and experimental work. T.L. was responsible for formal analysis, and manuscript review and editing. A.H. provided overall leadership overseeing conceptualization, methodology, validation, and formal analysis while also managing computational resources, conducting investigations, drafting the manuscript, supervising the research, and overseeing project administration.

Funding Open access funding provided by Ariel University. The authors did not receive support from any organization for the submitted work.

Data Availability All the analyzed data consists of publicly available benchmark datasets that are widely used for validating techniques in computer vision. Detailed references to these datasets are provided within the paper.

Declarations

Conflict of interest The authors declare no Conflict of interest.

Ethics Approval and Consent to Participate Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Chen, L., Zhu, Y., Papandreou, G., Schroff, F., & Adam, H. (2018). Encoder-decoder with atrous separable convolution for semantic image segmentation. CoRR **abs/1802.02611**

Cordts, M., Omran, M., Ramos, S., Rehfeld, T., Enzweiler, M., Benenson, R., Franke, U., Roth, S., & Schiele, B. (2016). The cityscapes dataset for semantic urban scene understanding. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 3213–3223.

Defossez, A., Bottou, L., Bach, F., & Usunier, N. (2022). A simple convergence proof of adam and adagrad. Machine Learning Research.

Diko, A., Avola, D., Cascio, M., & Cinque, L. (2024). ReViT: Enhancing vision transformers feature diversity with attention residual connections, https://arxiv.org/abs/2402.11301.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2020). An image is worth 16×16 words: Transformers for image recognition at scale. CoRR **abs/2010.11929**.

Duchi, J., Hazan, E., & Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research, 12*, 2121–2159.

Everingham, M., Eslami, S. M. A., Van Gool, L., Williams, C. K. I., Winn, J., & Zisserman, A. (2015). The pascal visual object classes challenge: A retrospective. *International Journal of Computer Vision,111*(1), 98–136.

Han, D., Kim, J., & Kim, J. (2017). Deep pyramidal residual networks. In *2017 IEEE conference on computer vision and pattern recognition (CVPR)*, pp. 6307–6315.

Howard, A., Sandler, M., Chen, B., Wang, W., Chen, L.-C., Tan, M., Chu, G., Vasudevan, V., Zhu, Y., Pang, R., Adam, H., & Le, Q. (2019). Searching for MobileNetV3 . In *IEEE/CVF international conference on computer vision (ICCV)* pp. 1314–1324.

Hu, J., Shen, L., & Sun, G. (2018). Squeeze-and-excitation networks. In *2018 IEEE/CVF conference on computer vision and pattern recognition*, pp. 7132–7141.

Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K.Q. (2017). Densely connected convolutional networks. In *2017 IEEE conference on computer vision and pattern recognition (CVPR)*, pp. 2261–2269.

Keskar, N.S., & Socher, R. (2017). Improving generalization performance by switching from adam to sgd. arXiv

Kingma, D.P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980

Krizhevsky, A. (2009). Learning multiple layers of features from tiny images. Technical report, 32–33.

Landro, N., Gallo, I., & Grassa, R.L. (2020). Mixing ADAM and SGD: A combined optimization method. CoRR **abs/2011.08042** arXiv:2011.08042

Li, S., Tian, J., Wang, Z., Zhang, L., Liu, Z., Jin, W., Liu, Y., Sun, B., & Li, S. Z. (2024). Unveiling the backbone-optimizer coupling bias in visual representation learning https://arxiv.org/abs/2410.06373

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft coco: Common objects in context. In *European conference on computer vision*, pp. 740–755. Springer.

Li, A., Sun, S., Zhang, Z., Feng, M., Wu, C., & Li, W. (2023). A multi-scale traffic object detection algorithm for road scenes based on improved yolov5. *Electronics, 12*(4), 878.

Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., & Guo, B. (2021). Swin transformer: Hierarchical vision transformer using shifted windows. CoRR **abs/2103.14030**

Loshchilov, I., & Hutter, F. (2017). Fixing weight decay regularization in adam. arXiv preprint arXiv:1711.05101, *5*:5

Luo, L., Xiong, Y., Liu, Y., & Sun, X. (2019). Adaptive gradient methods with dynamic bound of learning rate. CoRR arxiv:1902.09843

Mulloy, P. G. (1994). Smoothing data with faster moving averages. *Technical Analysis of Stocks & Commodities*.

Polyak, B. T. (1964). Some methods of speeding up the convergence of iteration methods. USSR computational Mathematics and Mathematical Physics.

Qian, N. (1999). On the momentum term in gradient descent learning algorithms. *Neural networks: the official journal of the International Neural Network Society, 12*(1), 145–151.

Reddi, S. J., Kale, S., & Kumar, S. (2019). On the convergence of adam and beyond. CoRR arxiv:1904.09237

Robbins, H., & Monro, S. (1951). A stochastic approximation method. *The annals of mathematical statistics*, 400–407.

Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A. C., & Fei-Fei, L. (2015). ImageNet large scale visual recognition challenge. *International Journal of Computer Vision (IJCV), 115*(3), 211–252. https://doi.org/10.1007/s11263-015-0816-y

Sadiq, M., Masood, S., & Pal, O. (2022). Fd-yolov5: A fuzzy image enhancement based robust object detection model for safety helmet detection. *International Journal of Fuzzy Systems*, **24**.

Tan, M., & Le, Q. V. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. CoRR **abs/1905.11946**.

Tieleman, T., & Hinton, G. (2012). Lecture 6.5—RmsProp: Divide the gradient by a running average of its recent magnitude. *COURSERA: Neural Networks for Machine Learning*.

Wu, H., Xiao, B., Codella, N., Liu, M., Dai, X., Yuan, L., & Zhang, L. (2021). Cvt: Introducing convolutions to vision transformers. In *2021 IEEE/CVF international conference on computer vision (ICCV)*, pp. 22–31.

Xie, X., Zhou, P., Li, H., Lin, Z., & Yan, S. (2024). Adan: Adaptive nesterov momentum algorithm for faster optimizing deep models. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 46*(12), 9508–9520. https://doi.org/10.1109/TPAMI.2024.3423382

Yao, Z., Gholami, A., Shen, S., Mustafa, M., Keutzer, K., & Mahoney, M. (2021). Adahessian: An adaptive second order optimizer for machine learning. *Proceedings of the AAAI Conference on Artificial Intelligence, 35*, 10665–10673.

Yu, W., Luo, M., Zhou, P., Si, C., Zhou, Y., Wang, X., Feng, J., & Yan, S. (2022). Metaformer is actually what you need for vision. In *2022 IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, pp. 10809–10819. https://doi.org/10.1109/CVPR52688.2022.01055

Yuan, L., Chen, Y., Wang, T., Yu, W., Shi, Y., Jiang, Z., Tay, F.E.H., Feng, J., & Yan, S. (2021). Tokens-to-token vit: Training vision transformers from scratch on imagenet. In *2021 IEEE/CVF international conference on computer vision (ICCV)*, pp. 538–547.

Yu, W., Si, C., Zhou, P., Luo, M., Zhou, Y., Feng, J., Yan, S., & Wang, X. (2024). Metaformer baselines for vision. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 46*(2), 896–912.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Roi Peleg[1,2] · Yair Smadar[1,2] · Teddy Lazebnik[3,4] · Assaf Hoogi[1,2]

✉ Assaf Hoogi
ahoogi@gmail.com

Roi Peleg
roi2307@gmail.com

Yair Smadar
yair.smadar1@gmail.com

Teddy Lazebnik
teddy.lazebnik@ju.se

1 School of Computer Science, Ariel University, Ariel, Israel

2 Data Science and Artificial Intelligence Research Center, Ariel University, Ariel, Israel

3 Department of Information Systems, Haifa University, Haifa, Israel

4 Department of Computing, Jonkoping University, Jonkoping, Sweden